

STATGR 5243 Final Project

End-to-End Machine Learning for NYC Airbnb Price Prediction

Yuxuan Ji (yj2924)

Pengyu Chen (pc3188)

Kexuan Liu (kl3745)

Sebastian Hoornweg (smh2289)

May 2026

1 Introduction

Accurate pricing is one of the central challenges in short-term rental markets. For Airbnb listings in New York City, nightly price is influenced by multiple layers of information at once: geographic location, room type, host scale, review activity, and listing-level positioning. These factors do not act independently, and many of them are unlikely to follow a simple linear pattern. That makes Airbnb pricing a good setting for an end-to-end machine learning project in which data cleaning, exploratory analysis, feature engineering, model selection, and communication all matter.

This project uses the *New York City Airbnb Open Data* dataset published on Kaggle [?]. The raw data contain 48,895 listings and 16 original variables describing listing characteristics, reviews, host activity, and geographic coordinates for New York City listings in 2019. Our primary goal is to build a supervised learning model that can predict listing price with reasonable accuracy, while also identifying the factors that contribute most to pricing differences across the city.

The project is organized around three questions. First, can we predict Airbnb prices in New York City with useful accuracy after cleaning and engineering the available features? Second, which variables appear to be the most important drivers of price? Third, when both linear and tree-based models are considered, which modeling approach offers the best

balance between predictive performance and practical usability?

The final workflow treats price prediction as a regression problem and uses the log-transformed nightly price as the response variable. After comparing tuned Elastic Net, Random Forest, and Gradient Boosting models, the report concludes that the two tree-based methods perform almost identically and clearly outperform the linear baseline. Although Gradient Boosting is marginally stronger on RMSE and R^2 in the held-out test set, the final deployed model is a tuned Random Forest because its performance is essentially tied, its feature importance is straightforward to interpret, and it is easier to justify as the model behind the dashboard deliverable.

2 Data Description and Evaluation

2.1 Data Source

The project uses the file `AB_NYC_2019.csv`, obtained from the Kaggle dataset *New York City Airbnb Open Data*. According to the dataset description, the data summarize Airbnb listing activity and listing-level metrics in New York City for 2019. The file contains 48,895 observations and 16 original variables, making it large enough for model development while still being compact enough to support transparent exploratory analysis and reproducible preprocessing.

2.2 Raw Data Structure

The raw dataset combines several types of information:

- Identifier fields such as `id` and `host_id`
- Geographic fields including `neighbourhood_group`, `neighbourhood`, `latitude`, and `longitude`
- Listing rule variables such as `room_type`, `minimum_nights`, and `availability_365`
- Review-related variables such as `number_of_reviews` and `reviews_per_month`
- A text field, `name`, that contains short listing descriptions
- A time field, `last_review`
- The target variable, `price`

The real-world data has high complexity and leaves us with significant data quality challenges:

- **Missing values:** `reviews_per_month` and `last_review` have over 20% missing entries.
- **Outliers:** `price` ranges from \$0 to \$10,000; `minimum_nights` reaches up to 1,250.
- **Mixed data types:** categorical (e.g., `neighbourhood`, `room_type`) and numerical (e.g., coordinates, review counts).
- **Unstructured text:** the `name` field contains descriptive keywords.
- **Strong spatial autocorrelation:** latitude and longitude define geographic patterns.

These characteristics require sophisticated cleaning, preprocessing, and feature engineering, justifying an **Advanced** level in data collection and preparation. The full code and datasets are publicly available at: <https://github.com/Darconshal/5243-Project-4>.

2.3 Data Quality Evaluation

The most obvious issue in the raw data is missingness. Both `reviews_per_month` and `last_review` have 10,052 missing values, or about 20.56% of the dataset. The text fields `name` and `host_name` also contain a very small number of missing entries, but those are minor compared with the review variables.

Table 1: Main Missing-Value Issues in the Raw Data

Variable	Missing Count	Missing Percentage
<code>reviews_per_month</code>	10,052	20.56%
<code>last_review</code>	10,052	20.56%
<code>host_name</code>	21	0.04%
<code>name</code>	16	0.03%

The second issue is the presence of extreme values. In the raw file, `price` ranges from 0 to 10,000, and `minimum_nights` ranges up to 1,250. Values in these tails are not useful for the short-term pricing problem targeted in this project and would distort both visual summaries and model training if left untreated.

2.4 Cleaning Strategy

The cleaning choices in the notebook follow a practical business-rule approach rather than purely statistical trimming.

- **Missing values:**

- `reviews_per_month` was filled with 0, as missing values indicate the listing never received a review.
- `last_review` was filled with a placeholder date (1900-01-01) to be transformed later into a temporal feature.

- **Outliers:**

- We applied business-rule filtering: prices must be $> \$0$ and $\leq \$500$ (representing plausible short-term rental rates).
- `minimum_nights` was capped at 30 days, removing unrealistic long-term listings.

- **Categorical inconsistencies:**

- All neighbourhood and borough names were stripped of extra whitespace and standardised to title case.

After outlier filtering, the working data contain 47,125 rows and the original 16 variables. After feature engineering and preprocessing preparation, the final modeling dataset contains 47,125 rows and 39 columns. The train-test split used in the notebook produces 37,700 training observations and 9,425 test observations.

Overall, the dataset is well suited to the project despite its imperfections. The data contain enough variation to support meaningful modeling, and the missing-value and outlier problems are substantial enough to justify a careful preprocessing pipeline.

3 Exploratory Data Analysis

We conducted an extensive EDA incorporating statistical summaries, visualizations, and unsupervised learning techniques to uncover latent structures.

3.1 Price Distribution

Figure 1 shows the distribution of nightly price after the business-rule filtering step. Even after restricting the data to listings with price no greater than \$500, the distribution remains clearly right-skewed. Most listings are concentrated in a moderate price range, while a smaller set of high-priced listings still stretches the upper tail.

This behavior supports the use of a log transformation for the target variable in later modeling. It also explains why comparing models only in raw price units would be misleading: much of the difficulty comes from capturing proportional price differences rather than absolute dollar gaps.

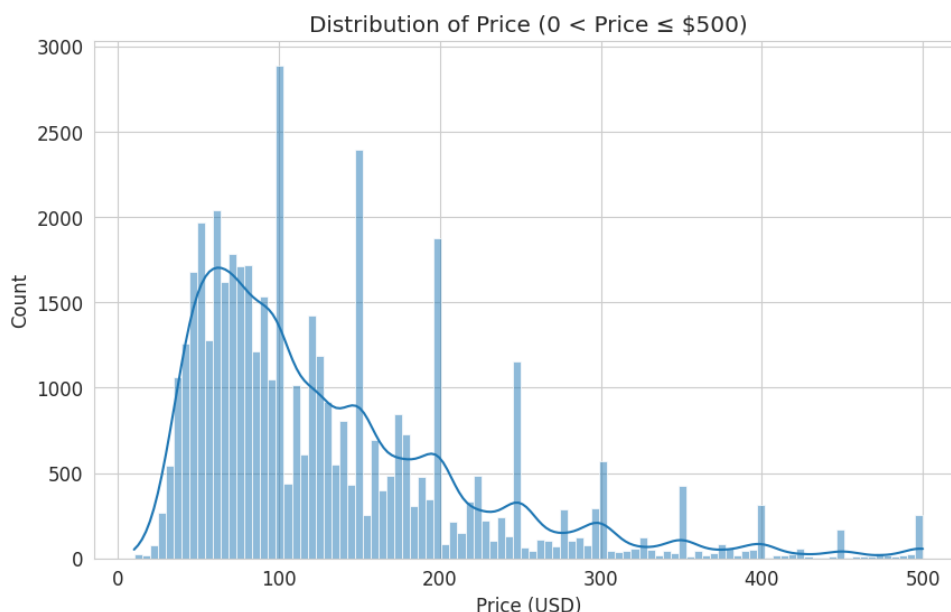


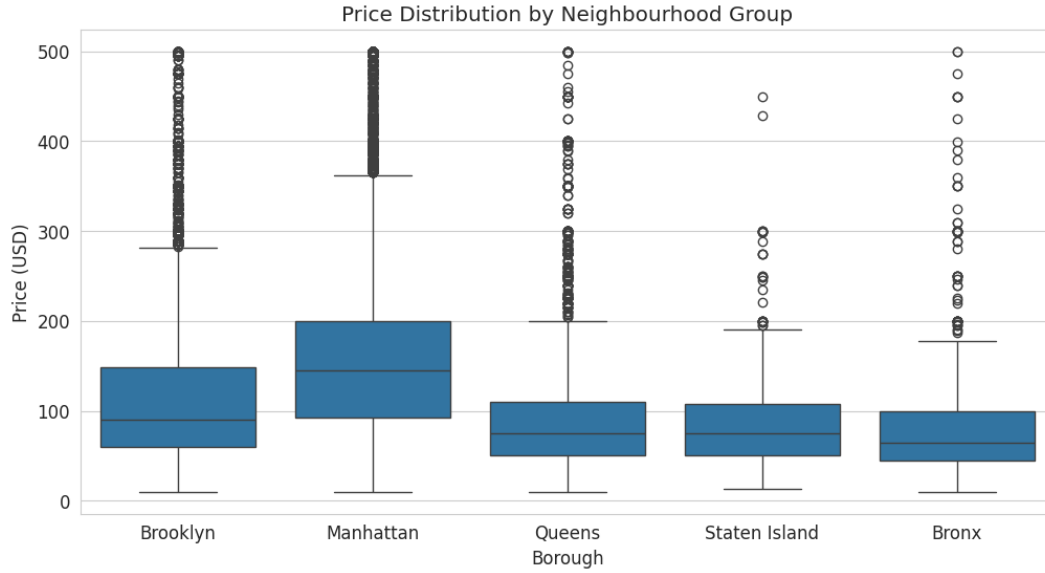
Figure 1: Distribution of nightly price after filtering to the range (0, 500].

3.2 Price by Borough and Room Type

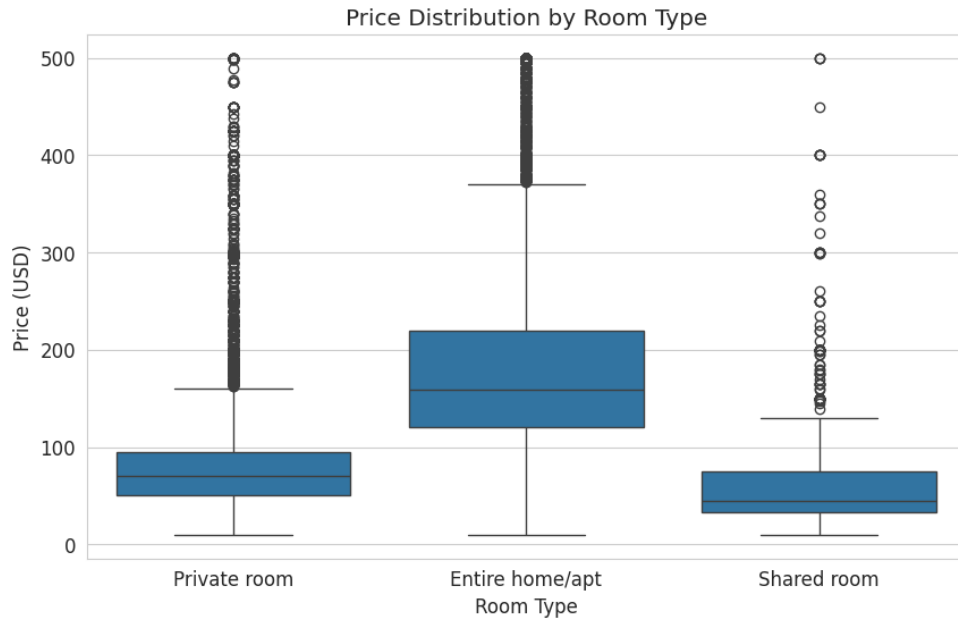
Price differences across boroughs and room types are substantial. The median listing price is highest in Manhattan (\$145), followed by Brooklyn (\$90), while Queens and Staten Island have median prices of \$75 and the Bronx has a median of \$65. Room type is even more sharply separated: the median price is \$159 for entire homes or apartments, \$70 for private rooms, and \$45 for shared rooms.

Table 2: Median Price by Borough and Room Type

Borough	Entire Home/Apt	Private Room	Shared Room
Bronx	100	53	40
Brooklyn	145	65	36
Manhattan	185	90	69
Queens	120	60	37
Staten Island	100	50	30



(a) Price distribution by borough.



(b) Price distribution by room type.

Figure 2: Distributional differences in price across major categorical groups.

These results strongly suggest that location and room type should remain central variables throughout the modeling pipeline.

3.3 Correlation Structure

The correlation heatmap in Figure 3 shows that the numeric relationships with price are not especially strong in simple linear terms. In the cleaned data, the strongest visible association with `log_price` is longitude, with a correlation of about -0.332 , while host listing count, latitude, and availability have weaker positive associations. This is an early indication that a linear model may capture only part of the signal and that tree-based methods are worth testing.

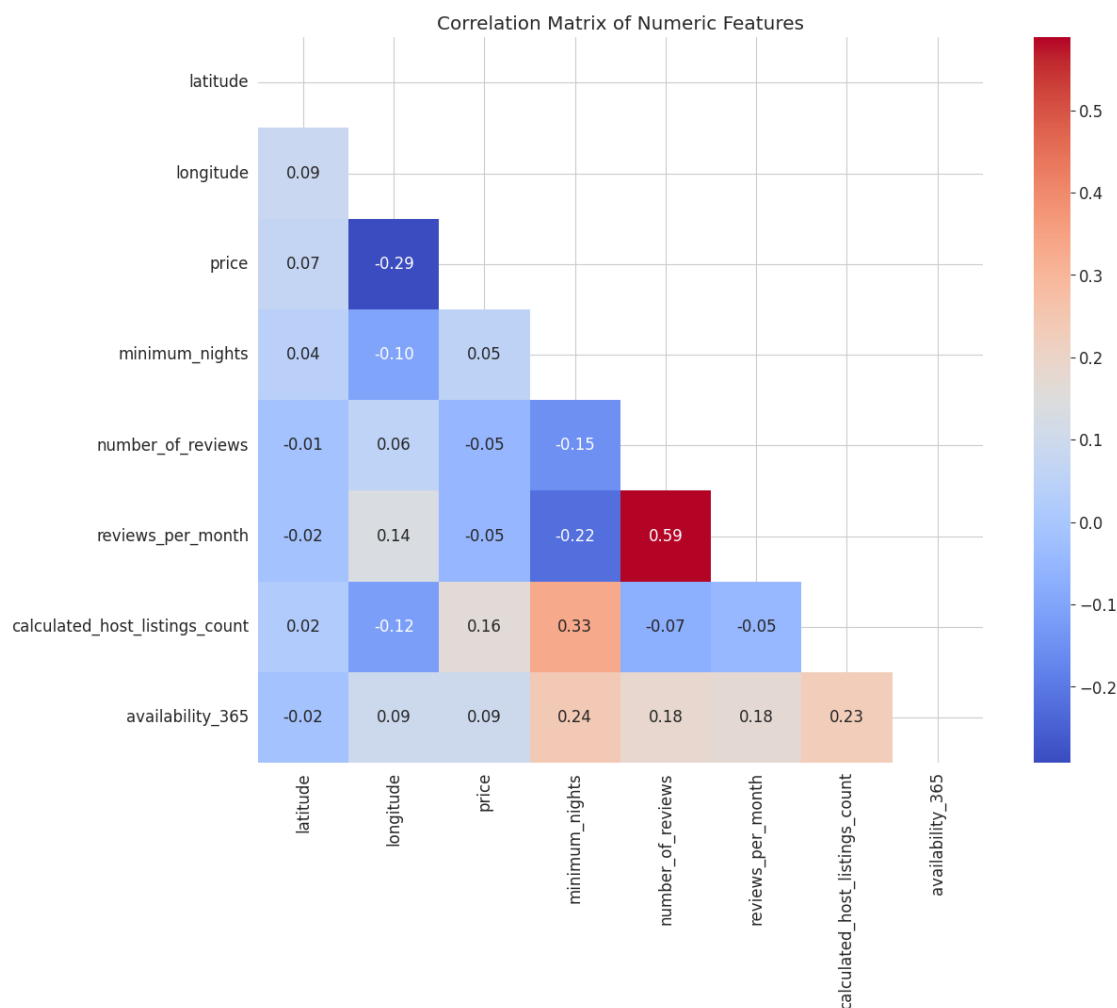


Figure 3: Correlation matrix of numeric variables in the cleaned data.

3.4 Geospatial Structure

A scatter plot of listings coloured by price revealed distinct high-price clusters in Manhattan and parts of Brooklyn. This spatial heterogeneity motivated geographic feature engineering.

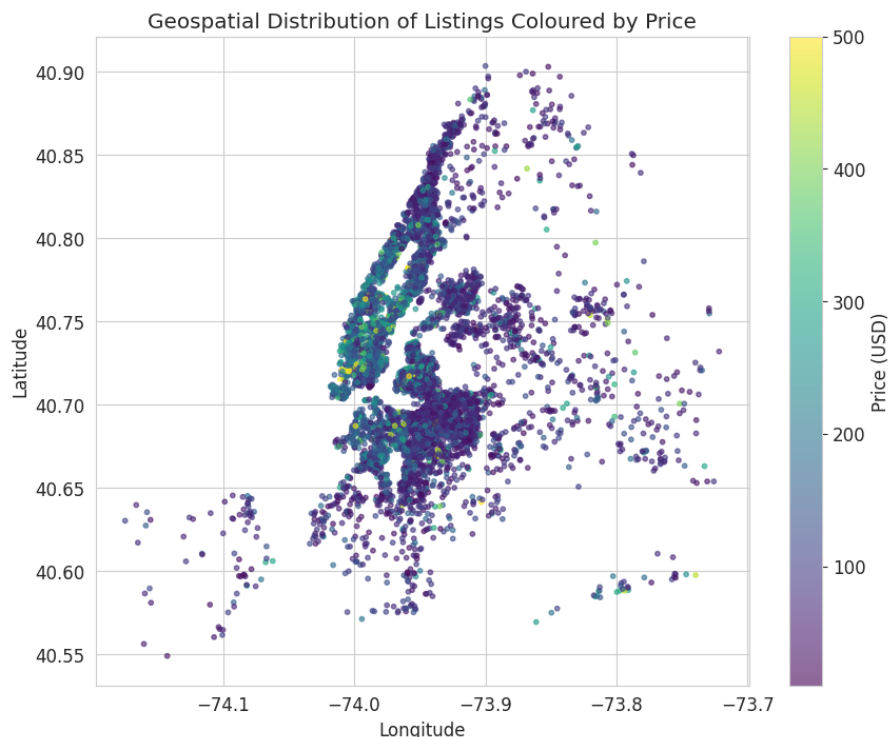


Figure 4: Sampled geospatial distribution of listings colored by price.

3.5 Unsupervised Structure

To represent local geographic structure more explicitly, the notebook applies K-means clustering with $k = 6$ to standardized latitude and longitude. The resulting clusters, shown in Figure 5, capture natural neighbourhood groupings that transcend administrative boundaries. Manhattan is divided into several sub-clusters, while Brooklyn and Queens form their own distinct regions.

The notebook also applies principal component analysis (PCA) to the main numeric variables. The first two principal components explain about 47.4% of the total variance, which is enough to show visible structure in a two-dimensional projection. Figure 6 suggests that a meaningful part of the overall variation is tied to location, availability, and host or review activity.

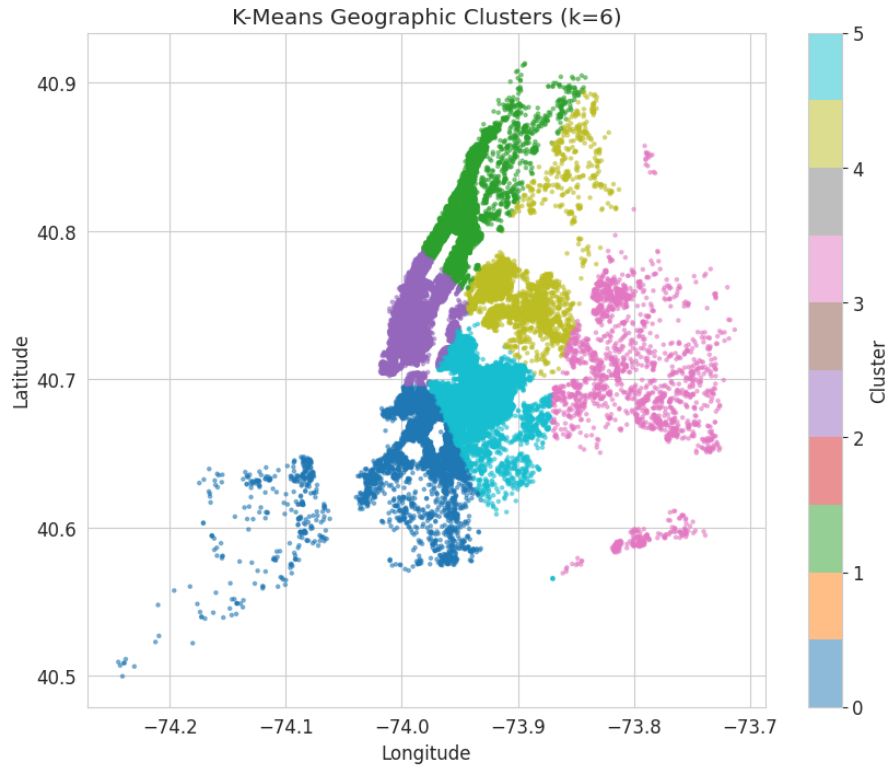


Figure 5: Geographic clusters from K-means on latitude and longitude ($k = 6$).

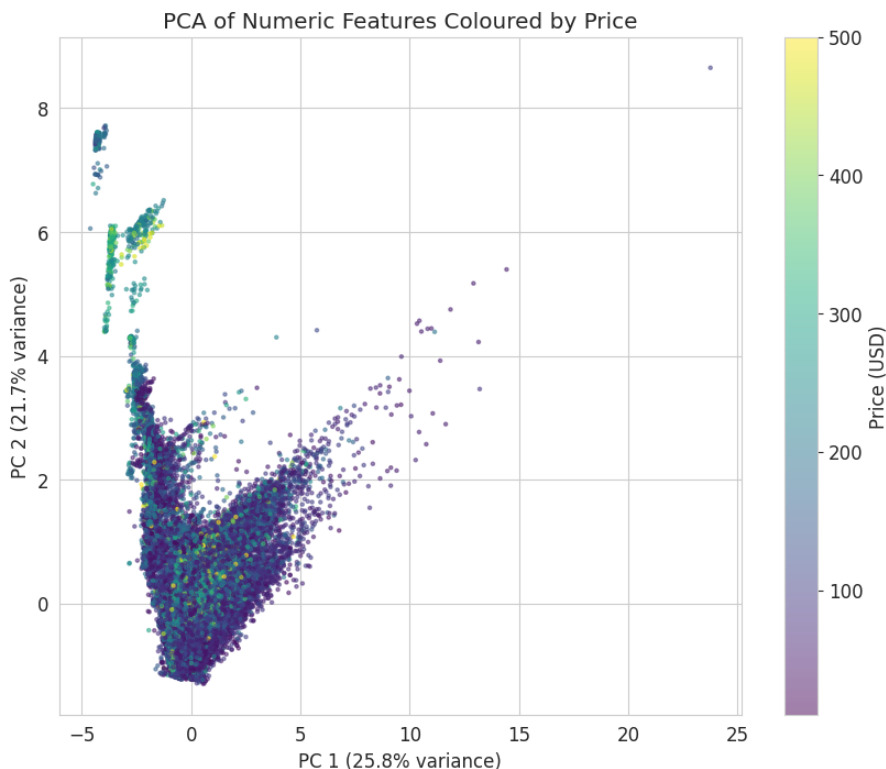


Figure 6: PCA projection of numeric features colored by price.

Taken together, the EDA supports four decisions used later in the report: model the target on the log scale, retain geographic coordinates and location categories, add engineered spatial features, and compare linear and nonlinear regression methods.

4 Feature Engineering and Preprocessing

Feature engineering is where the project moves from a standard tabular regression setup to a more tailored pricing model. Rather than relying only on the original columns, the notebook builds several groups of derived features intended to capture location quality, review dynamics, and listing positioning.

4.1 Text Features from Listing Name

From the `name` field, we extracted 13 keyword indicators including `luxury`, `cozy`, `spacious`, `private`, `studio`, `loft`, `park`, `view`, and `rooftop`. Each keyword was converted into a binary feature (e.g., `kw_luxury`). This NLP-light approach enriches the dataset with semantic signals.

4.2 Distance to Landmarks

We computed the **Euclidean distance** from each listing to four major NYC landmarks: Times Square, Central Park, Empire State Building, and JFK Airport. These features (`dist_times_sq`, `dist_central_park`, etc.) capture accessibility and tourism proximity.

4.3 Temporal Features

- `days_since_review` was created by computing the difference between a reference date (2019-07-01) and the `last_review` date. Values were capped at 3 years to handle never-reviewed listings.
- `has_reviews` was defined as a binary flag indicating whether `number_of_reviews` > 0.

4.4 Interaction and Ratio Features

- **Activity Index:** `number_of_reviews` $\times$ `reviews_per_month` — measures overall review activity.
- **Availability Ratio:** `availability_365`/365 — the fraction of the year the listing is available.
- **Host Listing Count (log):** $\log(1 + \text{calculated_host_listings_count})$ — reduces skew and reflects host experience.
- **Popularity Proxy:** `number_of_reviews` $\times$ `avail_ratio` — a combined measure of popularity and availability.

4.5 Target Transformation

The cleaned `price` was log-transformed to `log_price` to reduce skewness and stabilise variance, which is standard for economic pricing data.

4.6 Preprocessing Pipeline

A scikit-learn `ColumnTransformer` was built:

- Numerical features: median imputation + `StandardScaler`.
- Categorical features: constant imputation + `OneHotEncoder` (with `handle_unknown='ignore'`).

This pipeline was embedded within every model to ensure no data leakage and consistent transformation across training and test sets.

5 Model Development

5.1 Problem Setup

The final task is a supervised regression problem in which the response variable is `log_price`. The predictors include both original variables and engineered features. Model quality is evaluated on a held-out test set using RMSE, MAE, R^2 , and MAPE. RMSE on the log scale is treated as the main optimization metric during tuning.

5.2 Why These Models

Three models are considered:

- **Elastic Net:** a regularized linear model that provides a sensible baseline and works well with one-hot encoded categorical variables.
- **Random Forest:** a tree ensemble that captures nonlinear relationships and interactions automatically while remaining relatively robust and interpretable.
- **Gradient Boosting:** a boosting-based tree model that is often highly competitive on structured tabular data.

This set of models is intentionally balanced. Elastic Net provides a linear reference point, while Random Forest and Gradient Boosting test whether nonlinear structure materially improves prediction.

5.3 Training and Validation Design

The notebook uses an 80/20 train-test split, resulting in 37,700 training observations and 9,425 test observations. Hyperparameters are selected using 3-fold cross-validation. This choice keeps computation manageable while still providing a defensible validation scheme for a class project.

5.4 Hyperparameter Tuning

The search spaces are:

- **Elastic Net:** `alpha` $\in \{0.01, 0.1, 1\}$ and `l1_ratio` $\in \{0.1, 0.5, 0.9\}$
- **Random Forest:** `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{10, 20, \text{None}\}$, `min_samples_split` $\in \{5, 10\}$, and `min_samples_leaf` $\in \{1, 2\}$
- **Gradient Boosting:** `n_estimators` $\in \{100, 200\}$, `learning_rate` $\in \{0.05, 0.1\}$, and `max_depth` $\in \{3, 5\}$

The best tuning outcomes reported in the notebook are:

- Elastic Net: `alpha` = 0.01, `l1_ratio` = 0.1, best CV RMSE = 0.3881
- Random Forest: `n_estimators` = 200, `max_depth` = 20, `min_samples_split` = 10, `min_samples_leaf` = 1, best CV RMSE = 0.3635
- Gradient Boosting: `n_estimators` = 200, `learning_rate` = 0.1, `max_depth` = 5, best CV RMSE = 0.3608

These results already suggest that the two tree-based models are much more competitive than the linear baseline before the final test-set comparison is even considered.

6 Result Comparison and Final Selection

6.1 Test-Set Performance

Table 3 summarizes the final test-set performance reported in the notebook.

Table 3: Comparison of Tuned Models on the Test Set

Model	RMSE (log)	MAE (log)	R^2	MAPE
Tuned Gradient Boosting	0.3661	0.2772	0.6563	0.2902
Tuned Random Forest	0.3663	0.2751	0.6558	0.2894
Tuned Elastic Net	0.3937	0.2996	0.6025	0.3136

Two conclusions stand out immediately. First, the linear baseline is clearly weaker than either tree-based model. Second, Gradient Boosting and Random Forest are essentially tied.

Gradient Boosting has a marginal advantage on RMSE and R^2 , while Random Forest is slightly better on MAE and MAPE. The performance differences are so small that model choice cannot be justified by a single metric alone.

6.2 Why Random Forest Was Chosen as the Final Model

Although the test-set table places Gradient Boosting very slightly ahead on the primary metric, the final deployed model in the notebook is a tuned Random Forest. That choice is reasonable and should be explained explicitly in the report.

The main justification is that the two models are practically indistinguishable in predictive performance, so the final decision can be based on secondary considerations. In this project, Random Forest is the more convenient model for interpretation through feature importance, the more straightforward model to describe in a course report, and the more natural choice for deployment in the accompanying dashboard. In other words, the final selection is a *performance-near-tie decision*, not a contradiction of the numeric results.

6.3 Feature Importance

Figure 7 presents the top 25 feature importances from the final Random Forest model. The most influential predictors come from a few consistent groups: room type, geographic location, spatial clustering, host scale, availability, and landmark proximity. This pattern agrees with the earlier EDA and supports the view that pricing in this dataset is driven primarily by where a listing is, what kind of listing it is, and how the host operates it.

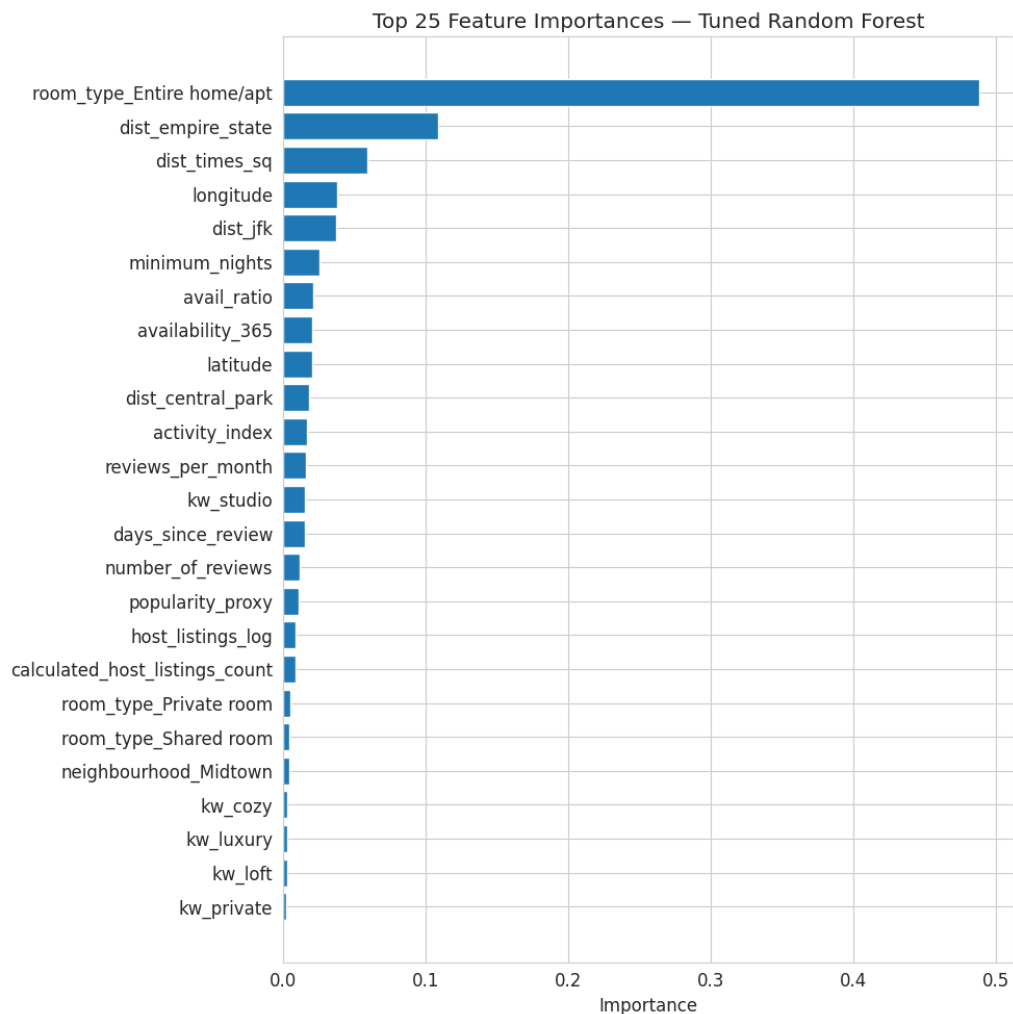


Figure 7: Top 25 feature importances from the final Random Forest model.

6.4 Residual Analysis

The residual diagnostics in Figures 8 and 9 suggest that the final model captures the overall structure of the problem reasonably well. The residual plot does not show a strong systematic pattern, and the residual distribution is centered around zero. The actual-versus-predicted scatter also tracks the 45-degree reference line fairly closely in the main part of the price range.

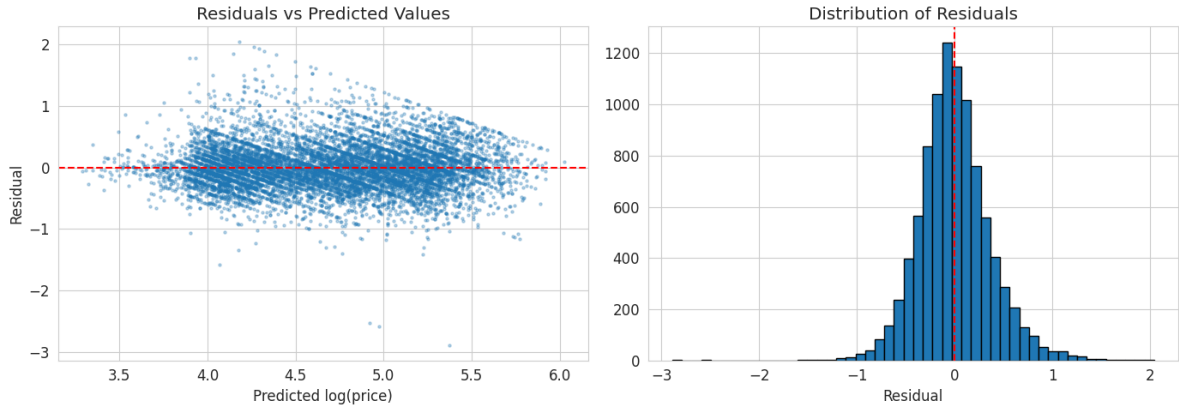


Figure 8: Residual plots for the final Random Forest model.

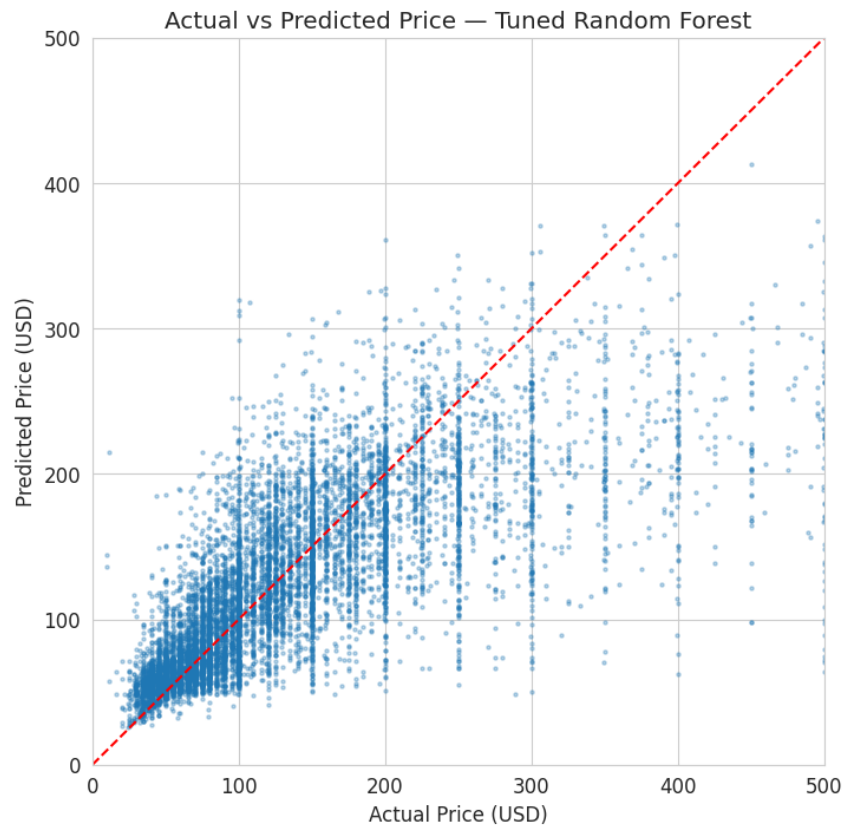


Figure 9: Actual versus predicted price in original dollar units.

As expected in a real pricing problem, predictions become noisier in the upper part of the price range. That does not invalidate the model, but it does suggest that the most expensive listings remain harder to explain using the available variables alone.

7 Interactive Dashboard

As a bonus deliverable, we developed an interactive web application using **Streamlit** that allows users to explore the data science workflow and predict Airbnb listing prices in real time. The dashboard is deployed locally and can be accessed by running the code from our GitHub repository.

7.1 Features

- **User Input Sidebar:** Users can adjust listing characteristics including borough, neighbourhood, room type, coordinates, minimum nights, review counts, availability, and keyword flags (e.g., `luxury`, `cozy`, `private`).
- **Real-Time Prediction:** On every parameter change, the pre-trained Tuned Random Forest model (loaded from `final_model.pkl`) instantly predicts the nightly price in US dollars, along with the corresponding log-price.
- **Feature Engineering Transparency:** The app automatically computes all engineered features (landmark distances, temporal flags, interaction terms) exactly as done during training, ensuring prediction consistency.
- **Input Summary:** A table displays the final feature vector fed into the model, helping users understand how their inputs translate into model variables.

7.2 How to Run

The dashboard can be launched locally by following these steps:

1. Clone the project repository: `https://github.com/Darconshal/5243-Project-4`
2. Install the required packages: `pip install streamlit pandas numpy scikit-learn joblib`
3. Navigate to the repository folder and run: `streamlit run streamlit_app.py`
4. Open `http://localhost:8501` in a web browser.

The application is fully self-contained and uses the same `final_model.pkl` file generated during model training.

The dashboard demonstrates a user-friendly, dynamic way to interact with a machine learning model, fulfilling the bonus communication criteria and enhancing the overall reproducibility and accessibility of the project.

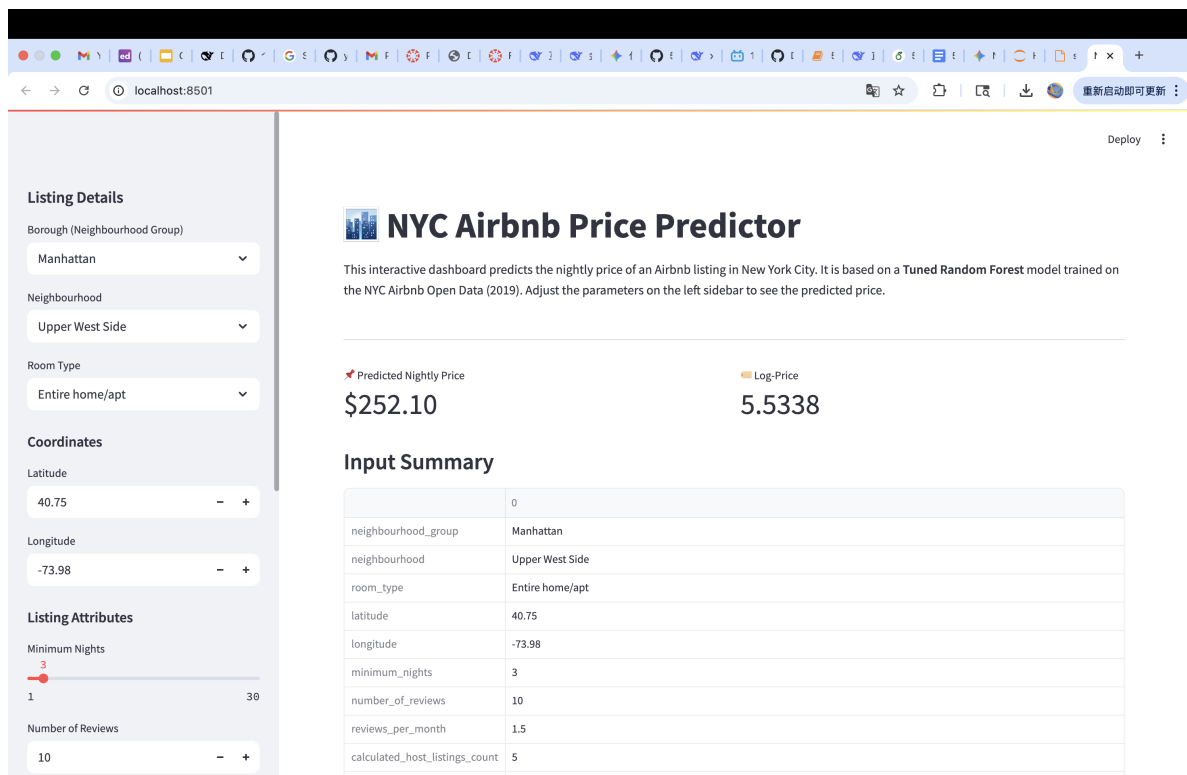


Figure 10: Streamlit dashboard interface showing input sidebar (left) and predicted price output (right).

8 Limitations and Future Work

The project performs well for a course-scale application, but several limitations remain.

First, the data cover only 2019. That means the model does not address how Airbnb pricing changes over time, how seasonal demand affects prices, or how the market may have shifted after 2019. A richer temporal dataset would make it possible to study price dynamics rather than just cross-sectional variation.

Second, the text component is intentionally lightweight. Listing names are reduced to a small set of keyword flags, which is useful for interpretability but leaves a large amount of language information unused. If listing descriptions, review text, or image data were available, the text and presentation side of the listing could be modeled much more fully.

Third, the spatial features are still approximations. The landmark variables use straight-line distance, which is simple and easy to interpret but not the same as real travel distance or neighborhood accessibility. A stronger future version could use travel-time or transit-based features instead.

Fourth, the deployed dashboard simplifies part of the training pipeline by using a default value for `geo_cluster`. For demonstration purposes, that is acceptable, but a production-quality implementation would apply the clustering transformation directly to the new input coordinates.

There are also several natural extensions on the modeling side. More advanced tabular learners such as XGBoost, LightGBM, or CatBoost may offer additional gains. Model interpretation could be improved with SHAP-based analysis rather than relying only on built-in feature importance. Finally, the dashboard could be extended to include uncertainty intervals, side-by-side scenario comparisons, or richer neighborhood-level summaries.

9 Conclusion

This project developed a full machine learning workflow for predicting Airbnb listing prices in New York City using the 2019 public dataset from Kaggle. The work moved from data-quality assessment and business-rule cleaning to exploratory analysis, feature engineering, preprocessing, model comparison, and an interactive dashboard.

The results show that tree-based models are much better suited to this task than a regularized linear baseline. Gradient Boosting and Random Forest performed almost identically on the held-out test set, and both substantially outperformed Elastic Net. The final project selected Random Forest because its predictive performance is effectively tied with Gradient Boosting while offering a cleaner path for interpretation and deployment.

Substantively, the project also identifies a clear set of price drivers. Room type, geographic location, local spatial structure, host listing volume, and listing availability all play important roles. These findings are consistent with intuition about the short-term rental market, but the project demonstrates them in a structured and reproducible way.

Taken together, the report shows that careful preprocessing and thoughtful feature engineering are at least as important as model choice alone. For this dataset, the strongest results come from combining domain-aware cleaning decisions with spatially informed features and a nonlinear model that can absorb their interactions.

10 Relative Information

10.1 Github Link

<https://github.com/Darconshal/5243-Project-4>

10.2 Team Contribution

Yuxuan Ji (yj2924): EDA Streamlit dashboard Creation

Pengyu Chen (pc3188) Model Development

Kexuan Liu (kl3745) Feature engineering

Sebastian Hoornweg (smh2289): Model Evaluation